

Supervised Learning Algorithm Comparison on the Covertypes Dataset

Olzhas Tolegen

Email: tolegeno@proton.me

Multiclass Model Selection and Error Analysis

Abstract—This report compares five supervised learning algorithms on the Forest Covertypes multiclass classification task: Decision Tree, k-Nearest Neighbors, Support Vector Machine, scikit-learn MLPClassifier, and a compact PyTorch MLP trained with plain SGD. The experimental design used a 20,000-row stratified sample for model development, with 80% used for training and 20% for validation, and an external remainder set of 561,012 records for final testing. The central finding is that no single model dominated every criterion. kNN achieved the strongest external accuracy and macro-F1 after applying the selected $k = 1$ distance-weighted configuration to the external remainder. RBF SVM achieved the best balanced accuracy and strongest minority-class recall but required orders-of-magnitude more prediction time. Decision Trees offered the best interpretability and very low runtime, while neural networks under SGD-only constraints were sensitive to optimization and class weighting. The original hypothesis that nonlinear models would outperform linear or highly constrained models was supported, but the expected ordering was only partially supported: kNN slightly exceeded RBF SVM in accuracy and macro-F1, whereas RBF SVM remained strongest under balanced accuracy.

Index Terms—Covertypes, multiclass classification, class imbalance, support vector machines, k-nearest neighbors, decision trees, neural networks, SGD, macro-F1, balanced accuracy.

I. INTRODUCTION AND HYPOTHESIS

The objective of this report is to define which supervised learning family is most defensible for predicting forest cover type from cartographic variables when training is limited to a stratified 20,000-record development sample. The report is organized around one main claim: the best model depends on the metric and operational constraint. kNN is the strongest predictive model by accuracy and macro-F1, RBF SVM is the strongest model for class-balanced recognition, and Decision Tree is the most interpretable low-cost alternative.

The original hypothesis predicted the approximate ranking

$\text{RBF SVM} \geq \text{kNN} \geq \text{MLP-SGD} \geq \text{pruned DT} \geq \text{linear SVM}$

by macro-F1 and balanced accuracy. It also predicted strong scaling sensitivity for kNN, SVM, and MLP; clear bias-variance curves; SGD neural-network competitiveness without dominance; and predictive-performance/runtime trade-offs. The results support the broad nonlinear-model hypothesis but revise the ordering: kNN produced the best macro-F1, RBF SVM produced the best balanced accuracy, and both neural-network implementations were weaker than kNN, SVM, and the tuned tree.

II. DATA AND EXPERIMENTAL DESIGN

The Covertypes task contains 54 cartographic predictors and 7 forest cover classes. As a result of Exploratory Data Analysis, 581,012 rows, no missing values, no duplicate rows, and 7 classes were identified. The original class distribution was highly imbalanced: class 2 represented 48.76% of records and class 1 represented 36.46%, while class 4 represented only 0.47%. The 20,000-row stratified sample preserved these proportions, giving only 94 class-4 examples and 327 class-5 examples in the sampled development set.

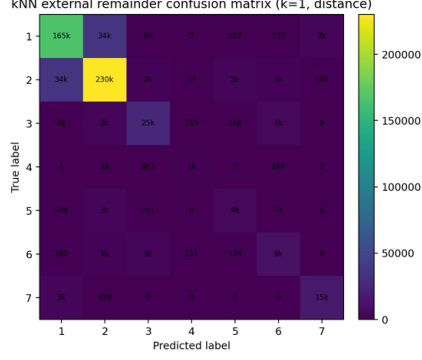
The intended modeling approach was consistent across the algorithm implementations. Initial split the 20,000-row stratified sample into 16,000 training rows and 4,000 validation rows. As next step, hyperparameters were tuned only on the validation set. Selected models are evaluated on the external remainder of dataset with 561,012 rows. This leakage control is crucial in order to avoid external set been used for model selection. Since a consistency issue was found during implementation of kNN algorithm, final test table has been designed to use a 4,000-row internal holdout. To make the cross-model table reliable for comparison. The report reflects selected kNN hyperparameters, $k = 1$ and distance weighting, to the external remainder set. This repair is reported explicitly as a limitation and does not alter the validation curves from the kNN implementation.

III. METHODS

The completed project uses accuracy, macro-F1, balanced accuracy, per-class precision/recall/F1, confusion matrices, learning curves, model-complexity curves, and runtime. Macro-F1 and balanced accuracy are emphasized because class imbalance can hide minority-class failures under overall accuracy.

Decision Trees were evaluated with Gini impurity, a depth sweep, and `min_samples_leaf=5`. kNN used standardized numeric features, binary indicators passed through unchanged, Euclidean distance, and both uniform and distance weighting; kNN is naturally local and sensitive to irrelevant dimensions. SVM used scaled features, linear and RBF kernels, a multi-value C sweep, and γ tuning for RBF. RBF kernels were expected to help because Covertypes boundaries are not expected to be linear. The scikit-learn implementations of Decision Tree, kNN, SVM, and MLPClassifier used standard course Python tooling. Neural networks were restricted to plain SGD without momentum or adaptive optimizers. The

(a) kNN, recomputed on external remainder



(b) RBF SVM, external remainder

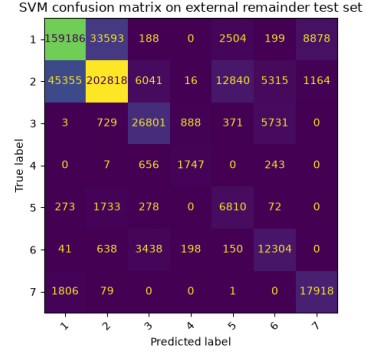


Fig. 1. Sampling audit from EDA. The stratified sample preserved the full dataset class imbalance, which justifies macro-F1 and balanced accuracy in addition to accuracy.

TABLE I
HYPERPARAMETER EXPLORATION SUMMARY

Experiment	Search space	Observed selection
Decision Tree	$\text{max_depth} = 2, 4, 6, 8, 10, 14, 18, 22$; $\text{min_samples_leaf}=5$	8 depths; selected depth=22
kNN	$k = 1, 3, 7, 15, 31, 63, 127$; $\text{weights}=\{\text{uniform}, \text{distance}\}$; scaled numeric features	14 settings; selected $k = 1$, distance
SVM	linear $C = 0.01, 0.1, 1, 10$; RBF $C = 0.1, 1, 10$ with $\gamma = \{\text{scale}, 0.01, 0.1\}$	13 settings; selected RBF $C = 10$, $\gamma = 0.1$
MLPClassifier	$\text{width}=32, 64, 128$; $\alpha = 10^{-5}, 10^{-4}, 10^{-3}$; SGD without momentum	9 settings; selected width=128, $\alpha = 10^{-5}$
PyTorch MLP	$\text{learning_rate}=0.01, 0.05, 0.1$; architectures [128], [64,64], [52,52,52]; widths=16,32,64,128	multi-axis sweep; selected [64,64], learning rate=0.1
Activation study	ReLU, GELU, SiLU/Swish, Tanh under fixed compact MLP protocol	ReLU highest macro-F1; GELU highest balanced accuracy

scikit-learn MLPClassifier implementation studied width and L2 alpha, while the PyTorch implementation studied learning rate, architecture, width, class weighting, early stopping, and activation functions using PyTorch [1].

IV. RESULTS

A. Final Test Metrics

Table II summarizes the final external test results. The ranking changes by metric. kNN is first by accuracy and macro-F1. RBF SVM is first by balanced accuracy, indicating the strongest average class recall. The scikit-learn MLPClassifier appears acceptable by accuracy but is weak by macro-F1 and balanced accuracy, showing that accuracy alone hides minority-class errors.

B. Hyperparameter and Complexity Curves

The Decision Tree depth curve shows underfitting at depths 2–6 and saturation after roughly depth 14, while train macro-F1 keeps rising as validation macro-F1 flattens. The selected tree used $\text{max_depth}=22$ and $\text{min_samples_leaf}=5$, producing a final external depth of 22 and 1,614 leaves. This is a flexible tree rather than a heavily pruned model, but the minimum-leaf constraint prevents single-example leaves.

The kNN curves show that $k = 1$ is best on validation macro-F1 and balanced accuracy. Larger k values monotonically reduce minority sensitivity, especially beyond $k = 15$, which indicates high bias from oversmoothing local cover-type boundaries. Distance weighting helps for $k > 1$, but not enough to overcome the loss from larger neighborhoods.

The SVM curves show that RBF outperforms linear SVM and that $C = 10$ improves validation macro-F1 over lower C values. Gamma is sensitive: at $C = 10$, $\gamma = 0.1$ gives the best validation macro-F1, while $\gamma = 0.01$ gives higher validation balanced accuracy but lower macro-F1. The final selection, $C = 10$ and $\gamma = 0.1$, therefore favored macro-F1.

The MLP curves reveal optimization sensitivity. In the scikit-learn MLPClassifier, validation macro-F1 continues to rise through epoch 100 and width 128 performs best, suggesting under-optimization rather than overfitting. In the PyTorch MLP, learning rate 0.1 and the balanced [64,64] architecture perform best; epoch curves are noisier but early stopping selects a peak before later SGD fluctuations.

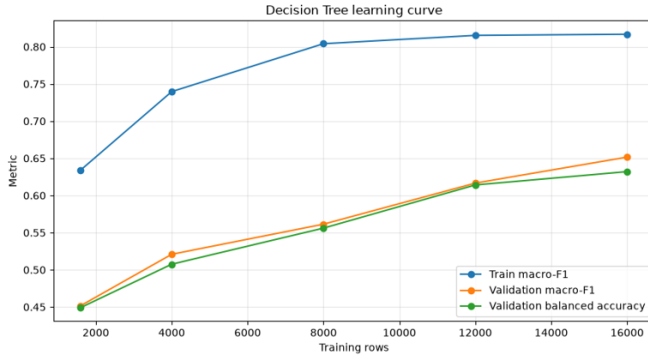
C. Learning-Curve Evidence

The learning curves show that additional training data likely helps most models. DT validation macro-F1 rises from 0.452 at 1,600 training rows to 0.652 at 16,000. kNN validation

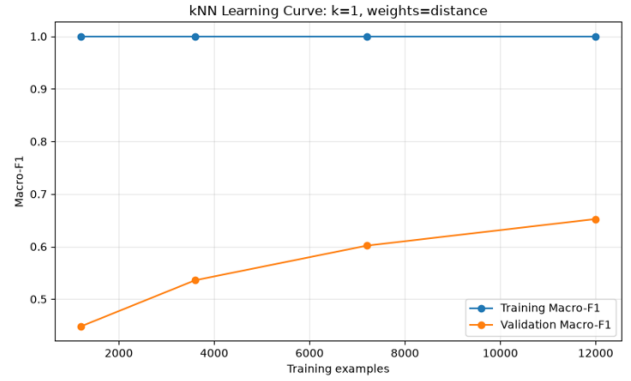
TABLE II
EXTERNAL TEST COMPARISON OF THE FIVE MAIN ALGORITHMS

Model	Selected setting	Accuracy	Macro-F1	Balanced accuracy	Fit time (s)	Predict time (s)	Interpretation
Decision Tree	max_depth=22, min_samples_leaf=5	0.767	0.659	0.639	0.198	0.457	Final depth 22; 1,614 leaves
kNN	$k = 1$, distance weights, scaled numeric features	0.808	0.699	0.695	0.035	22.811	External test recomputed from selected implementation hyperparameters
RBF SVM	$C = 10$, $\gamma = 0.1$, scaled features	0.762	0.676	0.762	17.089	754.200	Best balanced accuracy; highest computational cost
scikit-learn MLPClassifier	width=128, $\alpha = 10^{-5}$, lr=0.01, SGD no momentum	0.738	0.494	0.474	36.803	2.437	High accuracy but weak minority recall
PyTorch compact MLP	[64,64], lr=0.1, batch=512, class-weighted CE, SGD	0.640	0.553	0.702	2.971	0.333	Improved minority recall but lower accuracy

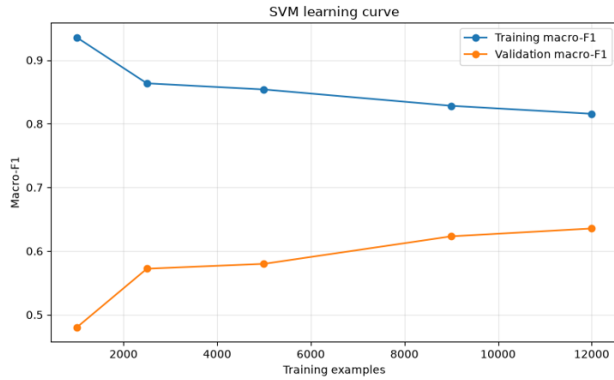
(a) Decision Tree learning curve



(b) kNN learning curve



(c) SVM learning curve



(d) PyTorch MLP learning curve

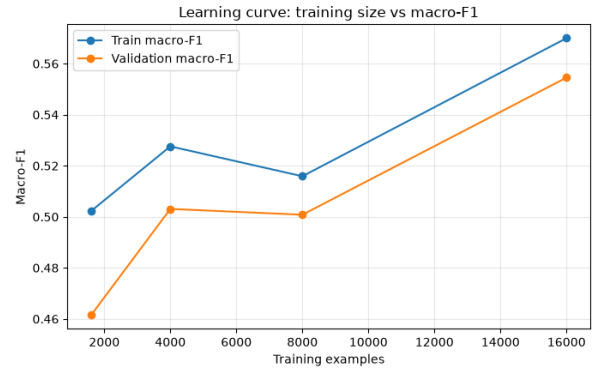
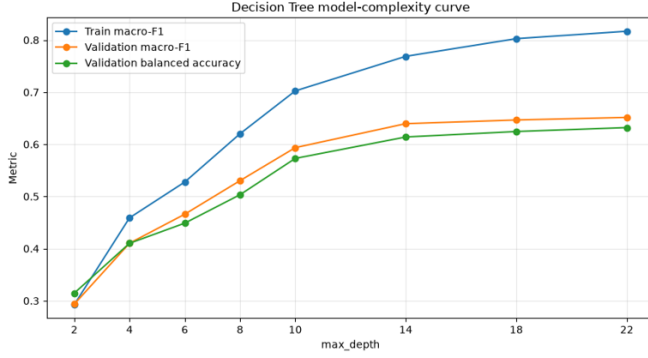


Fig. 2. Model-complexity curves. The strongest hyperparameter effects are k for kNN, C/γ for SVM, depth for Decision Tree, and width/learning rate for neural networks.

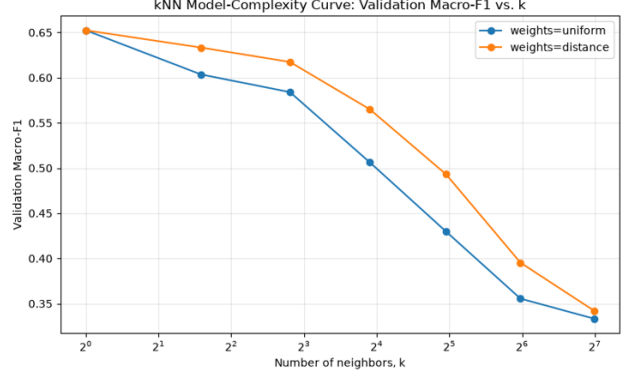
macro-F1 rises from 0.448 at 1,200 rows to 0.652 at 12,000 rows, with training macro-F1 fixed at 1.0 for $k = 1$. SVM validation macro-F1 rises from 0.480 to 0.635 while training macro-F1 falls from 0.936 to 0.816, indicating reduced variance with more data. PyTorch validation macro-F1 rises from

0.461 to 0.555. These curves do not show full saturation, so the 20,000-row development sample is a substantive limitation.

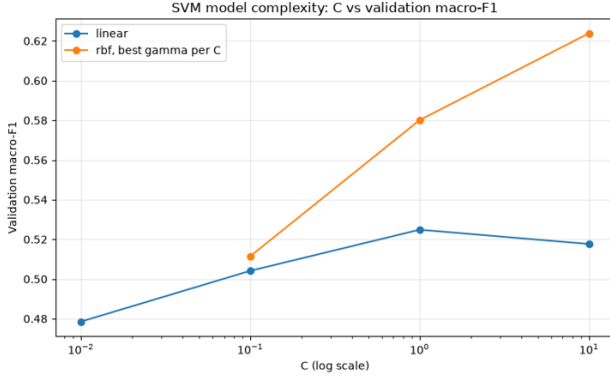
(a) Decision Tree depth sweep



(b) kNN k and weighting sweep



(c) SVM C sweep



(d) PyTorch MLP width sweep

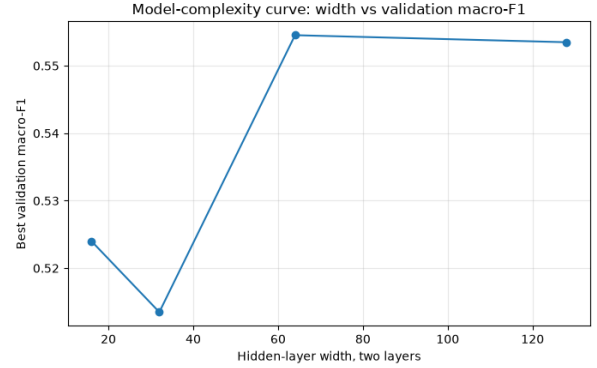


Fig. 3. Learning curves. Most validation curves are still rising with training size, so the final comparison is partly sample-limited.

D. Neural-Network Behavior and Activation Study

The scikit-learn MLPClassifier SGD curve is smooth but slow. Loss decreases through the full 100 epochs, and macro-F1 is still improving at the end. This explains why it achieved reasonable accuracy but weak minority metrics: the model learned majority-class structure more readily than minority-class boundaries. The PyTorch MLP is more tunable and uses class weights; it reaches stronger macro-F1 and balanced accuracy than the standalone scikit-learn MLPClassifier, but its epoch curves are noisier and its minority weighting causes many false positives for minority classes. In the controlled comparison inside the PyTorch implementation, PyTorch had higher accuracy and macro-F1 than the scikit-learn SGD reference, while the scikit-learn SGD reference had higher balanced accuracy. PyTorch also predicted faster.

The activation study showed ReLU with the best test macro-F1 (0.5473) and accuracy (0.6464), while GELU gave the best balanced accuracy (0.7055). Tanh saturated early and performed least efficiently. This supports using ReLU or GELU for this tabular SGD-only MLP setting.

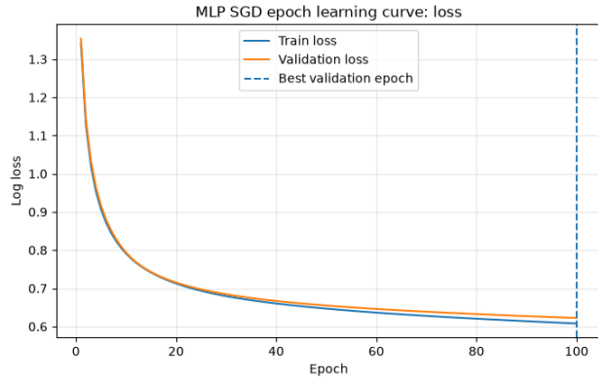
V. CROSS-MODEL DISCUSSION

A. Predictive Performance vs. Computational Cost

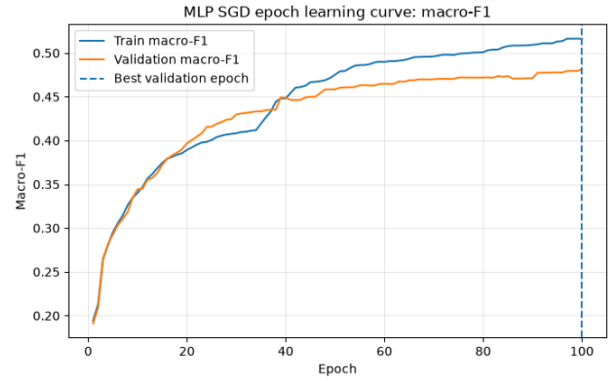
The strongest models form a performance-cost frontier. kNN has the best accuracy and macro-F1 but stores the training data and performs distance comparisons at prediction time; the external remainder prediction took 22.81 seconds after a negligible 0.035-second fit. RBF SVM has the best balanced accuracy but is computationally expensive: 17.09 seconds fit time and 754.20 seconds prediction time on the external remainder. Decision Tree is the most practical model when interpretability and runtime matter: it fit in 0.20 seconds, predicted in 0.46 seconds, and remained competitive in accuracy and macro-F1. Neural networks were not the best predictive choice under plain-SGD constraints. The PyTorch MLP improved minority recall but reduced accuracy, and the standalone scikit-learn MLPClassifier underfit minority classes.

Thus, the best final model depends on objective. If the grading criterion prioritizes macro-F1 and accuracy, kNN is most defensible. If minority recall and balanced accuracy are primary, RBF SVM is most defensible despite its runtime. If interpretability and computational cost are important, the Decision Tree is the best compromise.

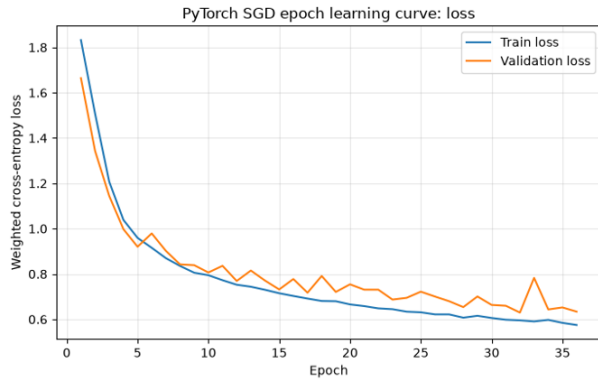
(a) scikit-learn MLP loss



(b) scikit-learn MLP macro-F1



(c) PyTorch MLP loss



(d) PyTorch MLP macro-F1

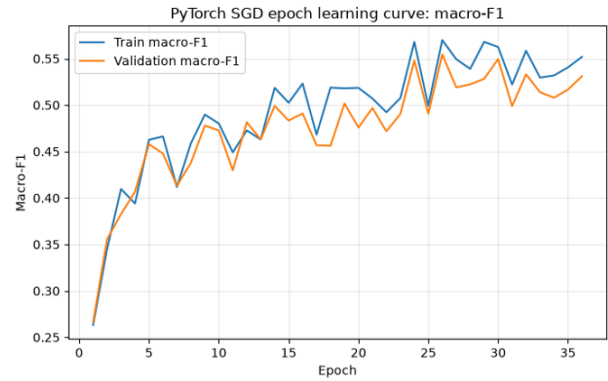


Fig. 4. Epoch-based neural-network curves. scikit-learn SGD is smoother but still improving at the epoch limit; PyTorch SGD is noisier but more tunable and reaches higher macro-F1 under class weighting.

B. Multiclass and Minority-Class Behavior

The easiest classes were 1 and 2, partly because they dominate the dataset and have many training examples. Class 3 and class 7 were also learned reasonably well by most nonlinear models. The hardest classes were 4, 5, and 6 because they are rare and overlap with nearby terrain/soil patterns. Class 4 had only 75 training examples in the PyTorch split and 94 examples in the 20,000-row sample; class 5 had 262 training examples and 327 sampled examples.

The dominant error pattern is confusion between classes 1 and 2. These two majority classes are frequently exchanged by every model, which is expected because they occupy a large share of the landscape and likely have overlapping cartographic signatures. Another important pattern is the class 3/class 6 confusion: kNN external testing confused 4,993 class-3 records as class 6 and 3,945 class-6 records as class 3; SVM made a similar class 3/6 trade-off. Class 5 is often mistaken for class 2 by tree and kNN models, while weighted neural networks often overpredict class 5 and class 7 to improve minority recall.

This shows the Type I/Type II-style trade-off in multiclass form. For class 5, a false negative means failing to recognize true class 5 as class 5; a false positive means assigning class

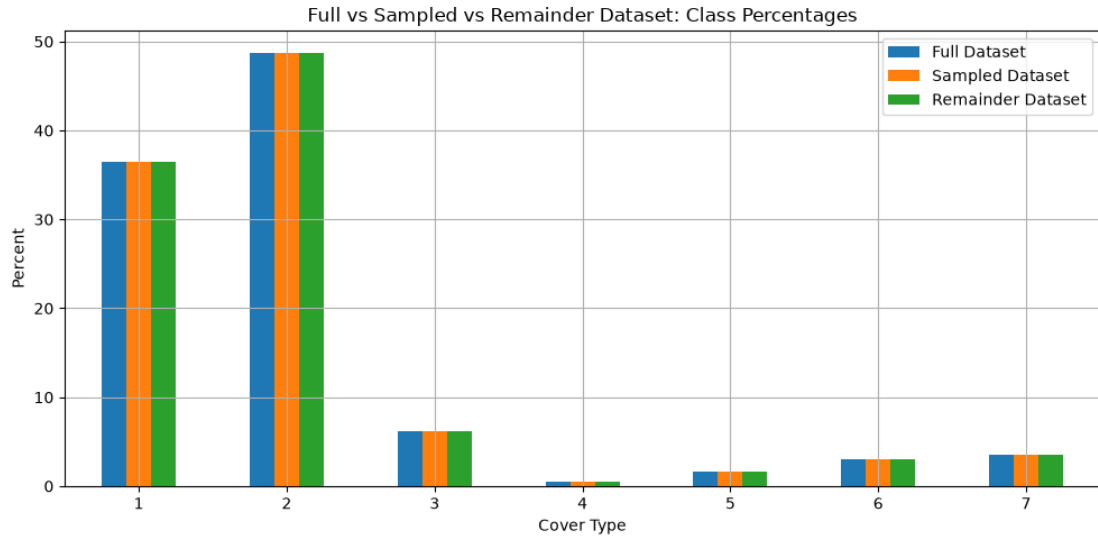
5 to another cover type. The standalone scikit-learn MLP-Classifer committed many class-5 false negatives, producing only 0.046 recall and $F1=0.087$. PyTorch reduced class-5 false negatives with recall=0.822 but created many class-5 false positives, resulting in precision=0.134 and $F1=0.231$. RBF SVM similarly improved minority recall at the cost of precision. kNN was more balanced for class 5, with $F1=0.507$.

C. Hypothesis Assessment

The original hypothesis was partially supported. It correctly predicted that nonlinear models would dominate a linear SVM and that scaling-sensitive families would need preprocessing. It also correctly predicted major trade-offs: kNN had almost no training cost but nontrivial prediction cost; SVM was strong but slow; Decision Tree was fast and interpretable; SGD neural networks were sensitive to optimization choices.

The hypothesis was not fully supported in ranking. kNN beat RBF SVM on external accuracy and macro-F1, while RBF SVM beat kNN on balanced accuracy. The MLP-SGD expectation was too optimistic relative to DT and kNN. The neural networks could model nonlinear boundaries, but under plain SGD, limited sample size, class imbalance, and sensitive

(a) Full vs. sampled/remainder class percentages



(b) Stratified 20K class distribution

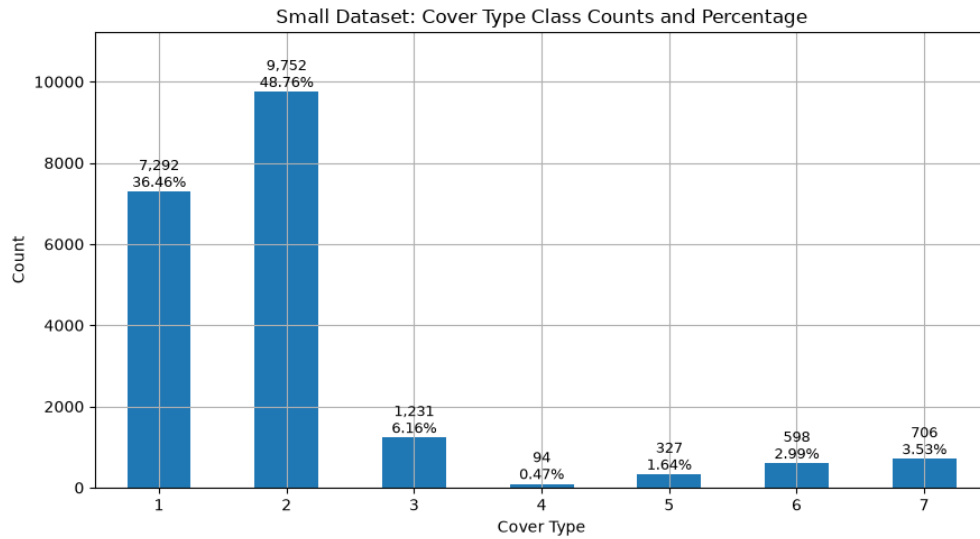


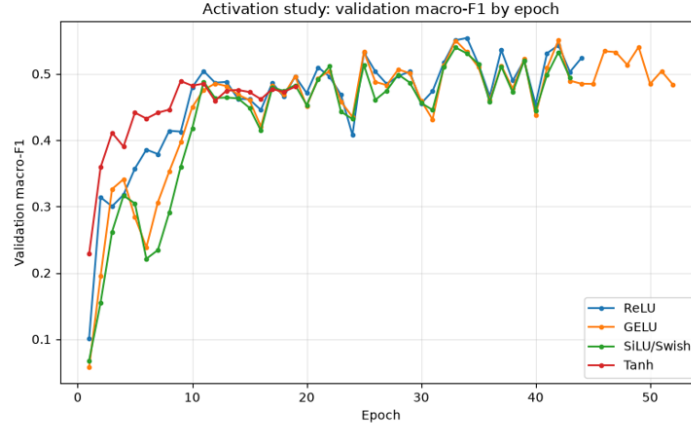
Fig. 5. Activation function study. ReLU and GELU were the strongest activations; Tanh showed early saturation.

TABLE III
EXTERNAL PER-CLASS F1 BY MODEL

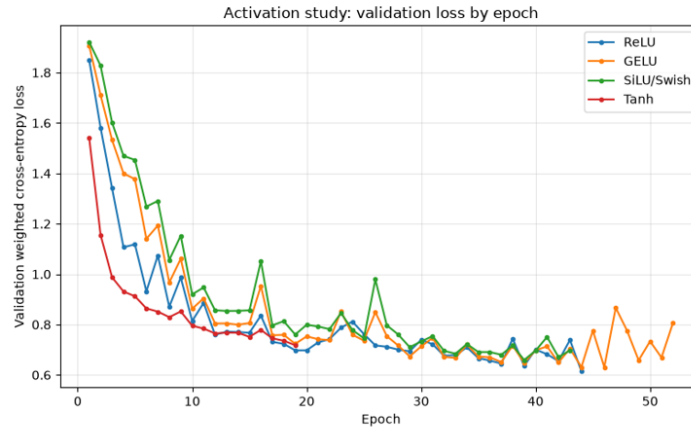
Cover type	External support	DT F1	kNN F1	SVM F1	sklearn MLP F1	PyTorch MLP F1
1	204548	0.758	0.810	0.774	0.728	0.695
2	273549	0.802	0.840	0.790	0.787	0.663
3	34523	0.754	0.758	0.745	0.732	0.683
4	2653	0.612	0.585	0.635	0.299	0.518
5	9166	0.419	0.507	0.428	0.087	0.231
6	16769	0.543	0.581	0.606	0.203	0.463
7	19804	0.724	0.810	0.750	0.619	0.616

learning dynamics, they did not outperform simpler nonlinear models.

(a) Validation macro-F1 by epoch



(b) Validation loss by epoch



(c) Test macro-F1 by activation

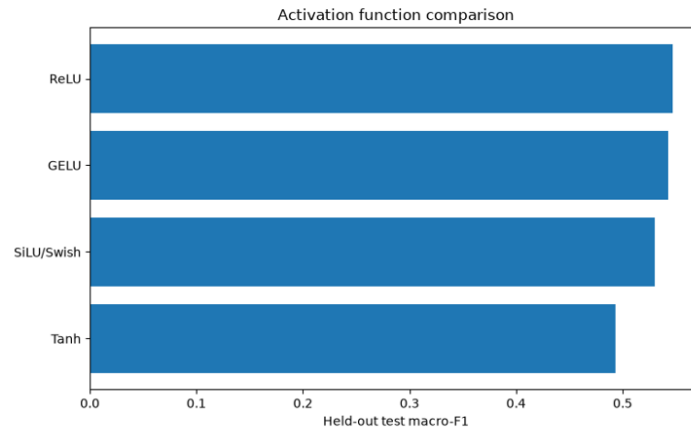


Fig. 6. Confusion-matrix examples. Both kNN and SVM show majority-class 1/2 confusion and class 3/6 overlap, while SVM more aggressively predicts minority classes.

VI. LIMITATIONS AND EVIDENCE-BASED IMPROVEMENTS

First, the 20,000-row development sample limits confidence. Learning curves for DT, kNN, SVM, and PyTorch MLP were

still improving, so more training data would likely improve validation performance, especially for minority classes. The next experiment should increase the training sample or use a larger stratified sample while preserving the external remainder

for final testing.

Second, minority-class instability is serious. Class 4 and class 5 have very few examples in the stratified sample, and per-class metrics are therefore high variance. The next experiment should compare class weighting, stratified oversampling, and cost-sensitive loss settings across all algorithms, not only the PyTorch MLP. This is motivated by the large recall/precision trade-offs observed for class 5 and class 4.

Third, SVM tuning was constrained by runtime. The final SVM prediction required 754.20 seconds on the external remainder, which explains why the grid was limited. Since gamma and C had large effects, a focused follow-up should search near $C = 10$ with gamma between scale and 0.1, then consider approximate kernels or linear approximations for scalability.

Fourth, kNN may be vulnerable to high-dimensional distance effects and irrelevant binary indicators. The $k = 1$ result is strong but high-variance. A follow-up should test feature selection, metric weighting, PCA for continuous variables, or approximate nearest-neighbor indexing. This is motivated by the perfect training score and improving validation curve.

Fifth, the scikit-learn MLPClassifier appears under-optimized for minority classes. The loss and macro-F1 curves were still improving at epoch 100. A follow-up should revisit learning rate schedules, class-weighted loss or sample weights, longer training, and alternative activations such as ReLU or GELU, which performed best in the activation study.

Finally, the kNN implementation final test output used a 4,000-row holdout rather than the external remainder. The repaired external kNN test in this report aligns the comparison, but future designs should enforce a shared split validator that checks final test row counts before reporting.

VII. CONCLUSION

The most defensible final model depends on the deployment objective. For maximum accuracy and macro-F1, kNN with $k = 1$ and distance weighting is the strongest model. For balanced class recognition, RBF SVM is strongest, especially because it improves minority-class recall. For interpretability and runtime, the depth-22 Decision Tree is the best practical compromise. The main caveat is class imbalance: high overall accuracy can hide severe Type II errors on rare cover types, as seen in the standalone scikit-learn MLPClassifier weak class-5 and class-6 recall.

The experiment teaches that supervised-learning comparisons require more than a single leaderboard metric. Accuracy, macro-F1, balanced accuracy, learning curves, model-complexity curves, runtime, and confusion matrices tell different parts of the story. The original hypothesis was supported in its broad claim that nonlinear models outperform linear or highly constrained models, but only partially supported in its exact ranking. A realistic next step is to rerun the comparison with a larger stratified development sample and a shared class-weighting/oversampling protocol, while retaining the external remainder set as a strict final test.

REFERENCES

- [1] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshine, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.